

# **M6800**

## **Microcomputer**

### **System Design Data**



**MOTOROLA**

# THE M6800 MICROCOMPUTER SYSTEM

The Motorola approach to practical microprocessor system implementation consists of:

- An interactive family of LSI semiconductor components with which to build microcomputer-based systems;
- A pervasive complement of design and evaluation tools;
- A growing line of peripheral equipment;
- An extensive library of functional and diagnostic software.

This comprehensive combination of hardware and software is augmented by a series of technical information manuals, a continuing program of customer training courses, and a team of systems application engineers that facilitate design and implementation of M6800-based systems.

## M6800 SEMICONDUCTOR COMPONENTS

The semiconductor components for the M6800 System are described in the following pages. Specific features of an M6800-based microcomputer design include:

- Single +5 volt power supply for system design simplicity and low cost
- Easily expandable architecture
- Repertoire of 72 powerful, variable-length instructions
- Three-state bus controls for maximum system flexibility
- Microcomputer family concept employing dynamically programmable/ bus compatible I/O functions
- 65K bytes of address capability
- High throughput, with clock rates to 1 MHz

## M6800 DESIGN AND EVALUATION TOOLS

### EXORciser

The EXORciser is a hardware/ software design, evaluation and diagnostic instrument. With its three basic modules, plus one or more of a series of optionally available plug-in accessory modules, it permits rapid emulation of any M6800-based microcomputer design. Built-in firmware (EXbug) allows the user to evaluate the performance of his software, the M6800 components, or the user's random logic designs. The EXORciser also provides the nucleus of a cost-effective system for development and debugging of M6800 software.

### Evaluation Module

The Evaluation Module provides a basic microcomputer built from M6800 components and board-mounted for convenient utilization. Each card contains all necessary components, except power supply, for the functional evaluation of M6800 components and permits running of user test programs via the bootstrap firmware designed into the unit.

### M6800 Design Evaluation Kit II

The MEK6800D2 Kit provides a useful and expandable tool for those who wish to develop systems with the MC6800 microprocessor without investing in expensive terminals. All parts needed to complete the system and get on the air are provided in the kit with the exception of the power supply. In addition to the expansion available on the basic microcomputer module, additional RAM, ROM, and I/O parts can be accommodated at a later date to implement more complex systems. Machine language programs can be entered through the system keyboard or via a built-in audio cassette interface system. Hexadecimal LED displays are provided for monitoring data and address information. A crystal-controlled clock generator is used to eliminate timing adjustments.

## M6800 PERIPHERAL EQUIPMENT

Trademarked EXORdisk and EXORTape, these two powerful peripherals are designed specifically for use in conjunction with the M6800 EXORciser. The versatile floppy disk (EXORdisk) is available with one to four disk drives and contains 256,256 bytes per diskette. The high-speed paper-tape reader (EXORTape) loads up to 250 characters per second into the EXORciser. This is 25 times faster than a standard ASR-33 teletype.

## M6800 SUPPORT SOFTWARE

Support Software for the M6800 is available in three forms to assist the user in generating the specific programs he ultimately wishes to implement with his M6800 system design:

**Resident Software** — designed to work in conjunction with the EXORciser to permit real-time development of system software in an emulated final microcomputer system. Provides the lowest program development cost where several M6800 microcomputer designs are contemplated over a period of time.

**Time-Sharing System** — provides a software package that permits immediate utilization of a variety of national and international time-share computers for development of the M6800 program. Recommended where maximum capability at low initial cost is required.

**In-House Computer System** — basic programs that permit M6800 system program development on in-house computers having FORTRAN IV capability and upward of 25K words of memory. Recommended for companies which are simultaneously running many large M6800 projects.

## M6800 SUPPORT LITERATURE

Key support documents which supplement the integrated circuit data sheets in this brochure include:

**M6800 Microprocessor Applications Manual** — a comprehensive 714-page manual describing in detail the operation and applications of the M6800 Family of microcomputer building blocks. Included are system organization, programming techniques, I/O and peripheral control techniques, family hardware characteristics, and system design and development. (\$25.00)

**M6800 Programming Manual** — a detailed 300-page guide defining the specific hardware and software properties of the MC6800 Microprocessor, and showing how to control the execution of its programmable instructions. Actual program examples are used to show how to build sequences and subroutines that perform useful (and often complex) functions. (\$10.00)

**M6800 Support Products Brochure** — description and data on the design and evaluation hardware, the peripheral equipment, and the software and firmware used with the M6800 Microprocessor system.

**Support Software Brochures** — a series of 6-page documents providing sample programs and editing features for use with the following support hardware and timesharing systems:

M6800 EXORciser

GE (General Electric Information Services International Network)

GE Background (Remote batch processing)

UCS (United Computing's Multiple Access Remote Computing Service)



# M6800

# Microcomputer

## System Design Data

### CONTENTS

|                                                                        | Page |
|------------------------------------------------------------------------|------|
| The M6800 Microcomputer Family. . . . .                                | 2    |
| MC6800 Microprocessing Unit (MPU). . . . .                             | 21   |
| MC6820 Peripheral Interface Adapter (PIA) . . . . .                    | 39   |
| MC6850 Asynchronous Communications Interface Adapter (ACIA). . . . .   | 49   |
| MC6852 Synchronous Serial Data Adapter (SSDA) . . . . .                | 57   |
| MC6860 0-600 bps Digital Modem. . . . .                                | 71   |
| MC6862 2400 bps Modulator . . . . .                                    | 87   |
| MC6870A, MC6871A,<br>MC6871B Two-Phase Microprocessor Clocks . . . . . | 95   |
| MC6880, MC8T26 Quad Three-State Bus Transceiver . . . . .              | 99   |
| XC6881, MC3449 Bi-Directional Bus Extender/Switch . . . . .            | 105  |
| XC6885-88, XC8T95-98 Hex Three-State Buffer/Inverters. . . . .         | 107  |
| MCM6810A 128 x 8-Bit Static Random Access Memory . . . . .             | 111  |
| MCM6830A 1024 x 8-Bit Read Only Memory . . . . .                       | 115  |
| MCM6832 2048 x 8-Bit Read Only Memory . . . . .                        | 119  |
| MCM68308 1024 x 8-Bit Read Only Memory . . . . .                       | 123  |
| MCM68316E 2048 x 8-Bit Read Only Memory . . . . .                      | 127  |
| MCM68708 1024 x 8-Bit Alterable Read Only Memory . . . . .             | 129  |
| MCM6604 4096-Bit Dynamic Random Access Memory . . . . .                | 131  |
| MCM6605A 4096-Bit Dynamic Random Access Memory . . . . .               | 139  |
| MPQ6842 MPU Clock Buffer . . . . .                                     | 153  |

EXORciser, EXbug, EXORdisk, and EXORTape are trademarks of Motorola Inc.

Second Printing  
©MOTOROLA INC., 1976

# THE M6800 MICROCOMPUTER FAMILY

## INTRODUCTION

The M6800 family of parts has been designed to set the standard for microcomputer system architectures. One of the initial goals was to minimize the number of required components and support packages. This was accomplished by designing the MC6800 Microprocessor with the total system problem in mind. All microprocessor systems require some form of static or dynamic memory. This requirement was fulfilled by the 1024 x 8-bit ROM (MCM6830AL) and the 128 x 8-bit RAM (MCM6810AL). The need for specialized data transfer functions was realized early, which led to the Peripheral Interface Adapter (MC6820) as well as several others which are in various design stages. The M6800 system is attractive to the system user because these peripheral parts appear to the microprocessor as simply memory locations on the address and data bus. They are completely programmable from the bus and their real time status is available through the bus. This architecture accomplished several things at the system level. First, it simplified the interface between memory and peripheral parts and, second, it eliminated the use for I/O instructions; both of these features increase system throughput.

Other attractive features of the M6800 family are that the output buffers are capable of driving standard TTL and only one power supply is required (+5 volts). The V<sub>GG</sub> power supply required by most MOS designs has been eliminated. This means the M6800 family can directly interface with standard TTL logic without the need for additional power supplies.

The intent of this specification is to define in detail the M6800 system architecture. This includes defining a mini-

um system and discussing requirements of complex systems. It also includes a discussion of the static and dynamic interaction between the microprocessor, memories and peripherals. This leads to static and dynamic specifications on the M6800 data bus and address bus. Further, it includes a discussion and specification of the microprocessor clocks and how they interact with the system and a definition of the microprocessor control lines such as Three-State Control, Halt, Normal Interrupt, Non-Maskable Interrupt and how they are implemented in a system environment. And finally, it includes a description of what occurs on the circuit interfaces during instruction execution.

## SYSTEM COMPONENT DESCRIPTIONS

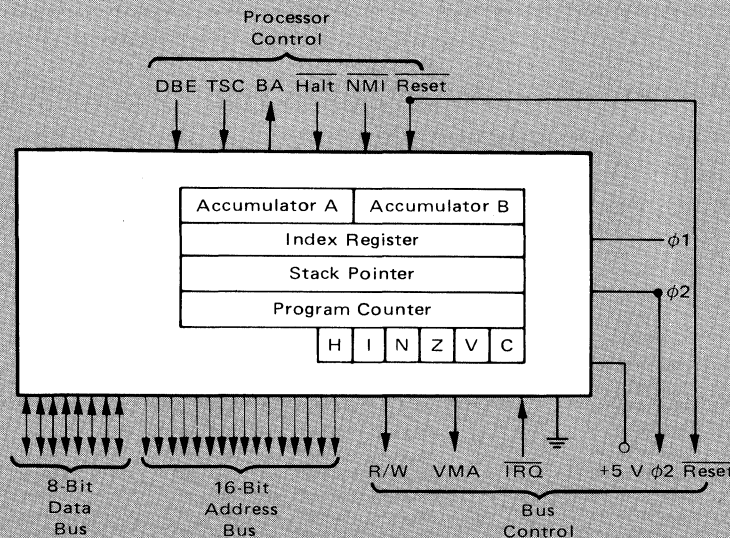
Before discussing the M6800 system, a general description of the MC6800 microprocessor, the 1024 x 8-bit ROM, the 128 x 8-bit RAM, the PIA and the ACIA is needed, as well as a detailed description of the microprocessor and peripheral input/output lines.

### MC6800 Microprocessor General Description

A symbolic diagram of the microprocessor and its input/output is shown in Figure 1. The processor is a bi-directional, bus-oriented, 8-bit parallel machine with 16 bits of address. For most systems, depending on interconnection capacitance, the processor is capable of directly interfacing with eight peripheral parts and one TTL load on the same bus at a 1 MHz minor cycle clock rate. For systems requiring additional peripheral parts, a Data Bus Extender (BEX) is available.

The processor has two 8-bit accumulators which are used to hold operands and results from the Arithmetic

FIGURE 1 — M6800 MICROPROCESSOR AND INPUT/OUTPUT LINES



Logic Unit (ALU). The 16-bit index register stores 16 bits of memory address for the index mode of memory addressing. The stack pointer is a two byte (8 bits/byte) register that contains the address of the next available location in an external push-down/pop-up stack. This stack is normally a random access read/write memory that may have any location (address) that is convenient. In those applications that require storage of information in the stack when power is lost, the stack must be non-volatile. The program counter is a 16-bit register that contains the program address. And finally, a condition code register (flag register) contains six bits of condition codes. The condition codes indicate the results of an ALU operation: Negative (N), Zero (Z), Overflow (V), Carry from bit 7 (C), and Half carry from bit 3 (H). These bits of the Condition Code Register are used as testable conditions for the conditional branch instructions. Bit 4 is the interrupt mask bit (I). The unused bits of the Condition Code Register (B6, B7) are always ones.

The minimum instruction execution time is 2 microseconds. Processor control lines include Reset, which automatically restarts the processor, as well as Interrupt Request and Non-Maskable Interrupt to monitor peripheral status. Finally, there is a Three-State Control, Data Bus Enable and a Halt control line which can be used for Direct Memory Access (DMA) or multiprocessing.

#### MCM6810A 128 x 8-Bit RAM General Description

The RAM is a static memory which interfaces directly to the MC6800 microprocessor. The RAM is organized in an 8-bit byte fashion.

The RAM has six Chip Select inputs, four active low and two active high, which interface directly to the address bus.

The RAM bus interface shown in Figure 2 demonstrates the simplicity of interface in the M6800 system. Since all M6800 components operate at the same TTL levels and with the same drive capability, the data, address, and control lines can be interconnected without adding external TTL buffers. Memory timing specifications have been set to permit simple operation at full speed with the microprocessor.

Four of the Chip Selects of the MCM6810A are used to decode the system address lines. In small and medium

sized systems, this address decoding will be sufficient to distinguish all packages in the system without using any additional address decoding packages.

#### MCM6830A 1024 x 8-Bit ROM General Description

The 8K ROM is a static memory which also interfaces directly to the MC6800 microprocessor. The output drivers are level compatible with the MC6800 family. The ROM is organized in an 8-bit byte fashion similar to the RAM. It has ten Address lines and four Chip Selects.

The ROM bus interface (Figure 3) is as straightforward as that of the RAM. All outputs may be connected directly to the data bus without drivers.

Three Chip Selects (mask programmable) on the MCM6830A are used to provide address decoding in the system. In many systems, the decoding possible with these lines will be sufficient to distinguish the ROM.

FIGURE 3 — MCM6830A ROM BUS INTERFACE

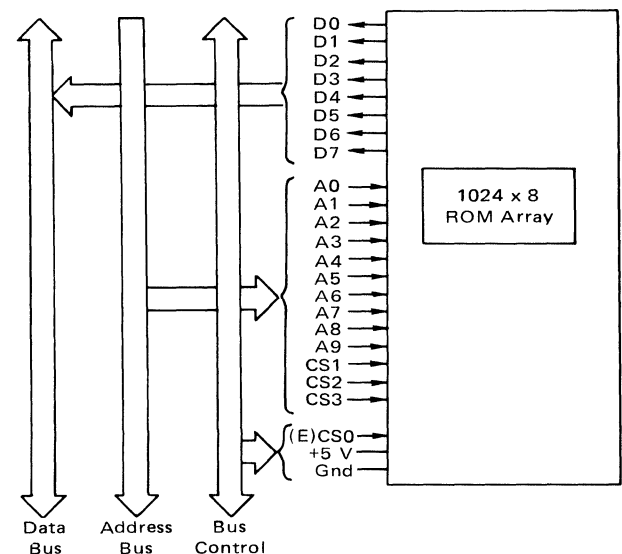
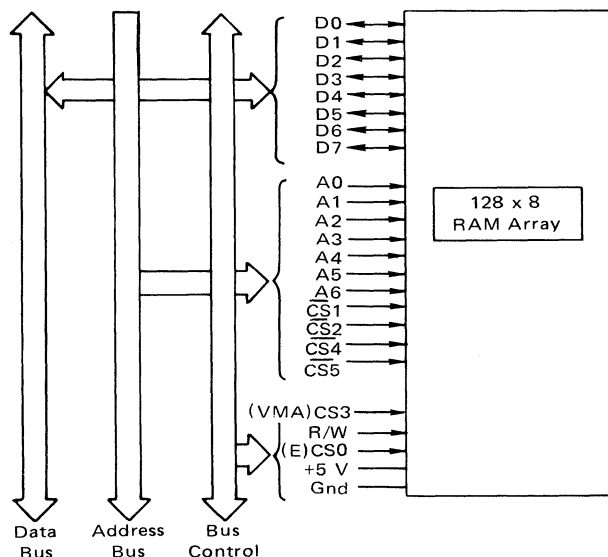


FIGURE 2 — MCM6810A RAM BUS INTERFACE



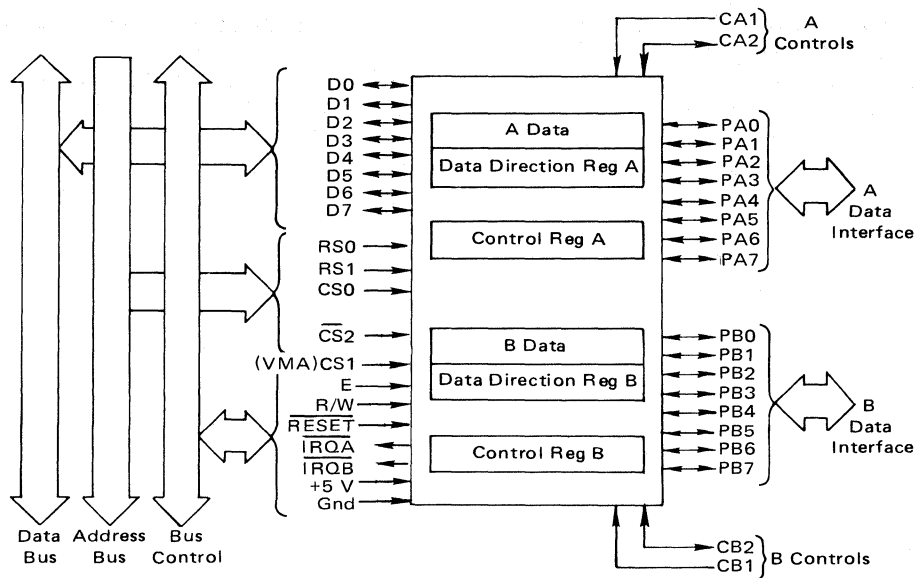
#### MC6820 Peripheral Interface Adapter (PIA) General Description

The MC6820 Peripheral Interface Adapter provides a universal means of interfacing peripheral equipment to the MC6800 Microprocessor through two 8-bit bi-directional peripheral data buses and four control lines. No external logic is required for interfacing to many peripheral devices.

The functional configuration of the PIA is programmed by the microprocessor during system initialization. Each of the peripheral data lines can be programmed to act as an input or output, and each of the four control/interrupt lines may be programmed for one of several control modes. This allows a high degree of flexibility in the overall operation of the interface.

Figure 4 shows the PIA system interface lines — the Data Bus lines D0-D7, Chip Selects CS0, CS1 and CS2, R/W, Enable E, the Register Selects RS0 and RS1, Reset, and the Interrupt Request lines IRQA and IRQB. The Data Bus lines, Chip Selects, Read/Write and Enable have the same static and dynamic characteristics as the other peripherals in the M6800 system.

FIGURE 4 – MC6820 PIA BUS INTERFACE



The Reset line is used to initialize the PIA. The Register Select lines RS0 and RS1 serve the same purpose in the PIA as address lines do in memory. They address the control and status registers, thereby making the PIA look like memory to the microprocessor.

#### MC6850 Asynchronous Communications Interface Adapter (ACIA) General Description

The MC6850 Asynchronous Communications Interface Adapter provides the data formatting and control to interface serial asynchronous data communications information to the MC6800 Microprocessor.

The parallel data of the bus system is serially transmitted and received (full-duplex) by the asynchronous data interface, with proper formatting and error checking. The functional configuration of the ACIA is programmed via the data bus during system initialization. A programmable Control Register provides variable word lengths, clock division ratios, transmit control, receive control, and interrupt control. Three I/O lines are provided to control external peripherals or modems. A Status Register is available to

the processor, and reflects the current status of the transmitter and receiver.

Figure 5 shows the ACIA system interface lines — the Data Bus lines D0–D7, Chip Selects CS0, CS1 and CS2, Read/Write, the Enable E, the Register Select RS, and the interrupt request output IRQ.

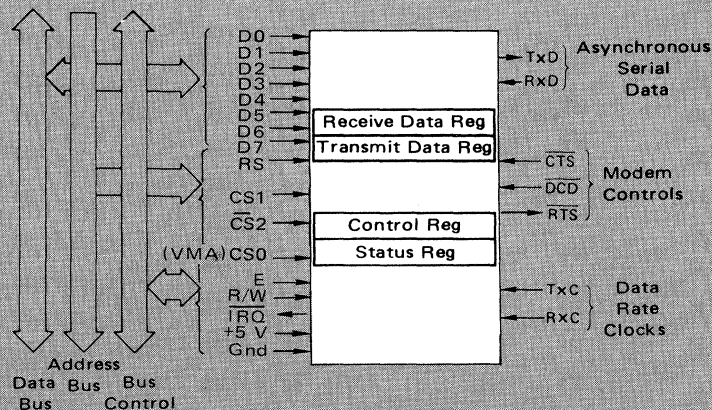
#### Other Peripherals

Other peripheral functions are at various stages of definition and design. All of these peripherals will be designed with the M6800 system concept in mind so that the same high degree of flexibility and compatibility is maintained.

#### MICROPROCESSOR INTERFACE LINES

The microprocessor input/output is broken into three groups: the bus interface lines, the bus control lines and the processor control lines. Description of bus interface and bus control lines are covered here. A description of the MPU control lines such as Halt and Data Bus Enable are covered later.

FIGURE 5 – MC6850 ACIA BUS INTERFACE



### Microprocessor Address Bus Lines (A0–A15)

Sixteen pins are used for the address bus. The outputs are three-state bus drivers. When the output is turned off, it is essentially an open circuit. This permits the MPU to be used in DMA applications. Putting TSC in its high state forces the address bus and R/W lines to go into the three-state mode.

### Microprocessor Data Bus Lines (D0–D7)

Eight pins are used for the data bus. It is bi-directional, transferring data to and from the memory and peripheral devices. The three-state drivers can be put into the three-state mode by forcing DBE low.

### Microprocessor Clock Inputs

#### Phase One and Phase Two ( $\phi_1$ , $\phi_2$ )

Two pins are used for a two-phase non-overlapping clock.

### Microprocessor Read/Write Line (R/W)

The Read/Write line is an output which signals the peripheral and memory devices whether the MPU is in a Read (high) or Write (low) state. The normal standby state of this signal is Read (high). Three-State Control going high will turn Read/Write to the high impedance state. Also, when the processor is halted, it will be in the high impedance state.

### Microprocessor Valid Memory Address Line (VMA)

This output indicates to peripheral devices that there is a valid address on the address bus. In normal operation, this signal should be utilized for enabling the RAM and peripheral interfaces such as the PIA and ACIA. This signal is active high. The output buffer is not three-state.

### Microprocessor Interrupt Request Line ( $\overline{\text{IRQ}}$ )

This level-sensitive input requests that an interrupt sequence be generated within the machine when  $\overline{\text{IRQ}}$  is low (Figure 13). The processor will wait until it completes the current instruction that is being executed before it recognizes the request. At that time, if the interrupt mask bit in the Condition Code Register is not set, the machine will begin an interrupt sequence. The contents of the Index Register, Program Counter, Accumulators, and Condition Code Register are stored away on the stack and the interrupt mask bit is set so no further interrupts may occur. At the end of the cycle, a 16-bit address will be placed on the address bus that points to a vectoring address which is located in memory locations FFF8 and FFF9. An address loaded at these locations causes the MPU to jump to an interrupt routine in memory.

The Halt line must be in the high state for interrupts to be serviced; interrupts will be latched internally while Halt is low. The wire-OR capability of the  $\overline{\text{IRQ}}$  input requires a 3 k ohm minimum external resistor to Vcc.

### PERIPHERAL AND MEMORY INTERFACE LINES

The peripheral and memory interface lines are the Data Bus D0–D7, the Address inputs, the Register Selects RS0–RS1, Chip Selects, R/W, the Enable E and Interrupt Request ( $\overline{\text{IRQ}}$ ). A description of these follows.

### Data Bus (D0–D7)

The peripheral Data Bus lines are bi-directional and capable of transferring data to and from the processor and peripheral devices. The drivers are three-state input-output buffers.

### Address Inputs (A0–A15)

Sixteen Address lines are available for addressing peripherals and memories. Seven are used for addressing the internal locations of the RAM and ten are used for addressing internal locations of the ROM. The address inputs are high impedance.

### Register Selects (RS0–RS1)

Register Select inputs on the peripherals such as the PIA and the ACIA are analogous to the Address inputs of the RAM and ROM. Bus address lines are tied directly to the Register Select inputs, in a minimum system configuration.

There are six locations within the PIA accessible to the microprocessor data bus: two Peripheral Registers, two Data Direction Registers, and two Control Registers. Selection of these locations is controlled by the RS0 and RS1 inputs together with CRA-2/CRB-2 in the Control Register, as shown in Table 1.

TABLE 1 – PIA INTERNAL ADDRESSING

| RS1 | RS0 | Control Register Bit |       | Location Selected         |
|-----|-----|----------------------|-------|---------------------------|
|     |     | CRA-2                | CRB-2 |                           |
| 0   | 0   | 1                    | X     | Peripheral Register A     |
| 0   | 0   | 0                    | X     | Data Direction Register A |
| 0   | 1   | X                    | X     | Control Register A        |
| 1   | 0   | X                    | 1     | Peripheral Register B     |
| 1   | 0   | X                    | 0     | Data Direction Register B |
| 1   | 1   | X                    | X     | Control Register B        |

X = Don't Care

For a functional description of these registers, see the MC6820 PIA data sheet.

The Register Select input (RS) performs a somewhat different function in the ACIA than in the PIA. The state of RS in the ACIA in conjunction with R/W determines which of four registers will be read by the microprocessor or written into by the microprocessor, as shown in Table 2:

TABLE 2 – ACIA INTERNAL ADDRESSING

| RS | R/W | Register               |
|----|-----|------------------------|
| 0  | 0   | Control Register       |
| 0  | 1   | Status Register        |
| 1  | 1   | Receive Data Register  |
| 1  | 0   | Transmit Data Register |

For a functional description of these registers, see the MC6850 ACIA data sheet.

### Chip Selects (CS0–CS5)

Chip Selects on the peripheral and memory devices are used to distinguish one device from another. The number of Chip Selects available varies from three on the PIA to six on the RAM. The Chip Selects may tie directly to the microprocessor address bus and VMA line in a minimum system configuration. They are high impedance inputs. The peripheral devices are enabled by various combinations of "true" and "not true" Chip Selects.

### Read/Write (R/W)

The Read/Write is a high impedance input which is used to control the direction of data flow through the processor data bus interface. When R/W is high (MPU read cycle)

the peripheral data bus drivers are turned on and the selected location is read. When R/W is low the data bus drivers are turned off and the MPU writes into the selected location.

#### Enable (E)

The Enable input E is the peripheral enable signal. The E input is a high impedance input which enables the peripheral output data buffers. One of the Chip Selects is used for the E enable on the RAM and ROM.

#### Peripheral Interrupt Request ( $\overline{\text{IRQA}}$ and $\overline{\text{IRQB}}$ )

The only difference between the special function peripherals (the ACIA and PIA) and memories from a system input/output pin consideration is that the peripheral parts have interrupt outputs which are used as a request for servicing.

$\overline{\text{IRQA}}$  and  $\overline{\text{IRQB}}$  from the PIA are OR tied to the system  $\overline{\text{IRQ}}$  line. Since the PIA may be used to detect incoming interrupt signals on any of its control lines, this connection must be made in order to initiate the interrupt sequence at the processor. The  $\overline{\text{IRQ}}$  will be pulled down by the PIA following detection of an active transition on any control line which has been enabled as a system interrupt.  $\overline{\text{IRQ}}$  will be held low until the interrupt is serviced. Thus no interrupts will be lost to the system even if the interrupt mask is set at the processor.

The ACIA interrupts the microprocessor under conditions that differ from those described for the PIA. Assuming the ACIA transmitter and receiver interrupts are enabled by bits in the control register, the ACIA will interrupt the MPU if the transmitter data register is empty, the receiver data register is full or if the Data Carrier Detect input goes high indicating a loss of the Modem carrier.

## STATIC AND DYNAMIC CHARACTERISTICS

Now that the microprocessor family interface has been defined, it is necessary to define the static and dynamic characteristics. Also, the microprocessor clocks, maximum ratings, power supply tolerances, temperature ranges, and test conditions will be defined. After these definitions are complete one will be able to put together a system and discuss interaction.

### Static Specifications

Table 3 shows the static specifications covering the microprocessor and peripherals. The table specifies clock levels, output levels, leakages, and capacitance in one table, making all the static information available for easy use.

### Dynamic Properties of the Data and Address Bus

Figure 6 and Table 4 show the dynamic characteristics of the Data and Address Bus interface. Address, Chip Selects R/W and VMA are all valid at  $t_{AS}$  (address setup time). The microprocessor then puts the address out at  $t_{AD}$  (address delay time).

The read access time  $t_{ACC} = t_{AS} + t_{DDR}$ , with  $t_{DDR}$  being the read data delay time. The peripheral data setup time is specified by  $t_{DSW}$ . The processor then puts data out at  $t_{DOW}$  (write data delay time). Hold time for Data Bus transfers is specified by  $t_H$ .

The interrupt request release time is  $t_{IR}$ . The Interrupt Request output driver in the peripheral parts is an open drain device. A pull up resistor is required on the  $\overline{\text{IRQ}}$  line. In a minimum system, the open drain devices are all tied together and then tied to the  $\overline{\text{IRQ}}$  input of the microprocessor. An interrupt is sensed by the microprocessor when  $\overline{\text{IRQ}}$  is low.

**TABLE 3 — STATIC CHARACTERISTICS** ( $V_{CC} = 5.0 \text{ V} \pm 5\%$ ,  $V_{SS} = 0$ ,  $T_A = 0$  to  $70^\circ \text{C}$  unless otherwise noted.)

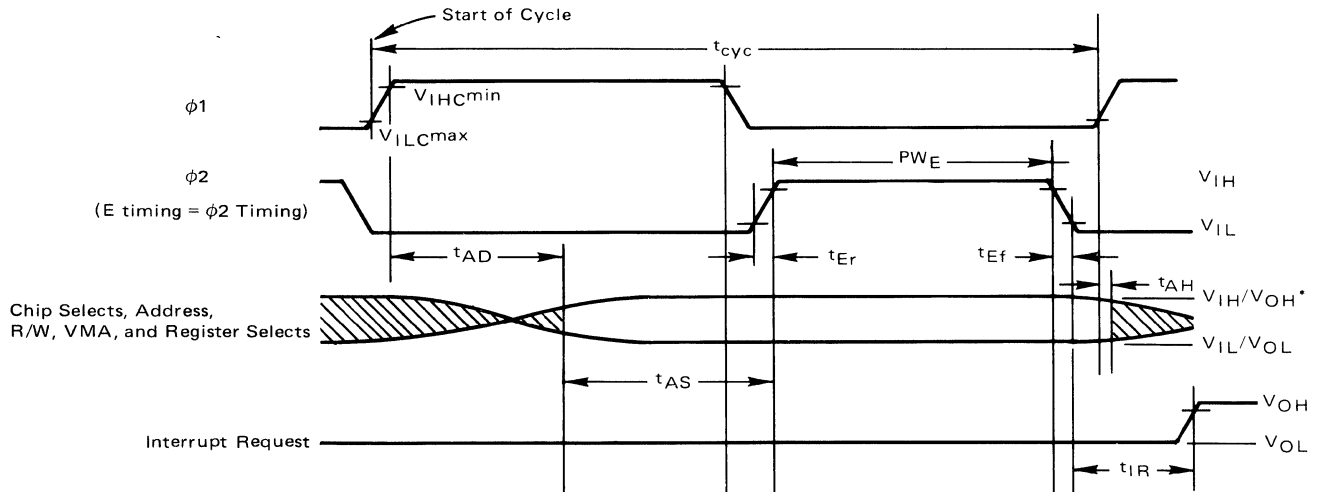
| Characteristic                                                                                                                                                                            | Symbol                | Min                                   | Typ                                               | Max                                                | Unit            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|---------------------------------------|---------------------------------------------------|----------------------------------------------------|-----------------|
| Input High Voltage — All Inputs except MPU $\phi 1$ and $\phi 2$<br>MPU $\phi 1$ and $\phi 2$                                                                                             | $V_{IH}$<br>$V_{IHC}$ | $V_{SS} + 2.0$<br>$V_{CC} - 0.3$      | —<br>—                                            | $V_{CC}$<br>$V_{CC} + 0.1$                         | Vdc             |
| Input Low Voltage — All Inputs except MPU $\phi 1$ and $\phi 2$<br>MPU $\phi 1$ and $\phi 2$                                                                                              | $V_{IL}$<br>$V_{ILC}$ | $V_{SS} - 0.3$<br>$V_{SS} - 0.1$      | —<br>—                                            | $V_{SS} + 0.8$<br>$V_{SS} + 0.3$                   | Vdc             |
| Input Leakage Current ( $V_{in} = 0$ to $5.25 \text{ Vdc}$ )<br>( $V_{CC} = 5.25 \text{ Vdc}$ ) All Inputs except MPU $\phi 1$ and $\phi 2$<br>( $V_{CC} = 0$ ) MPU $\phi 1$ and $\phi 2$ | $I_{in}$              | —<br>—                                | 1.0<br>—                                          | 2.5<br>100                                         | $\mu\text{Adc}$ |
| Three-State (Off State) Input Current<br>( $V_{in} = 0.4$ to $2.4 \text{ V}$ , $V_{CC} = 5.25 \text{ Vdc}$ ) D0-D7<br>A0-A15, R/W                                                         | $I_{TSI}$             | —<br>—                                | 2.0<br>—                                          | 10<br>100                                          | $\mu\text{Adc}$ |
| Output High Voltage<br>(Load A of Figure 8, $V_{CC} = 4.75 \text{ Vdc}$ )                                                                                                                 | $V_{OH}$              | $V_{SS} + 2.4$                        | —                                                 | —                                                  | Vdc             |
| Output Low Voltage<br>(Load A of Figure 8, $V_{CC} = 4.75 \text{ Vdc}$ )                                                                                                                  | $V_{OL}$              | —                                     | —                                                 | $V_{SS} + 0.4$                                     | Vdc             |
| Output Leakage Current, $\overline{\text{IRQ}}$ of Peripherals ( $V_{in} = 2.4 \text{ Vdc}$ )                                                                                             | $I_{LOH}$             | —                                     | 1.0                                               | 10                                                 | $\mu\text{Adc}$ |
| Capacitance*<br>( $V_{in} = 0$ , $T_A = 25^\circ \text{C}$ ,<br>$f = 1.0 \text{ MHz}$ )                                                                                                   | C                     | 80<br>—<br>—<br>—<br>—<br>—<br>—<br>— | 120<br>—<br>7.0<br>6.5<br>6.0<br>10<br>3.0<br>6.0 | 160<br>15<br>10<br>8.5<br>7.5<br>12.5<br>5.0<br>10 | pF              |
| $\phi 1$ and $\phi 2$ Overshoot/Undershoot — Input High Level<br>— Input Low Level                                                                                                        | $V_{OS}$              | $V_{CC} - 0.5$<br>$V_{SS} - 0.5$      | —<br>—                                            | $V_{CC} + 0.5$<br>$V_{SS} + 0.5$                   | Vdc             |
| Clock Overlap Voltage (Figure 7)                                                                                                                                                          | $V_{OV}$              | —                                     | —                                                 | 0.5                                                | Vdc             |

\*Capacitances are periodically sampled rather than 100% tested.

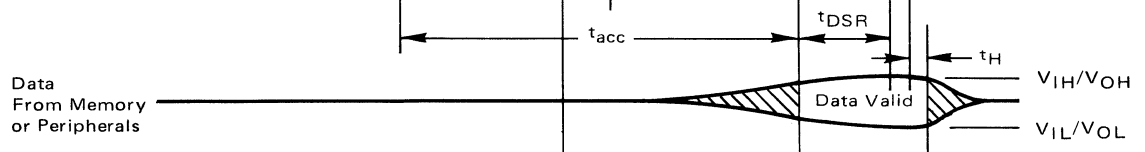


FIGURE 6 – BUS INTERFACE TIMING

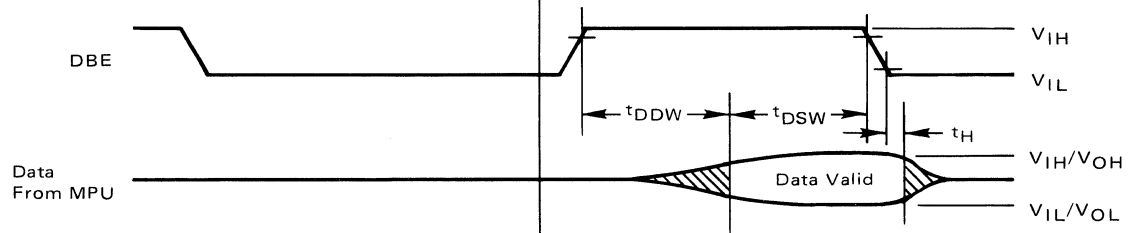
Address and Interrupt Timing



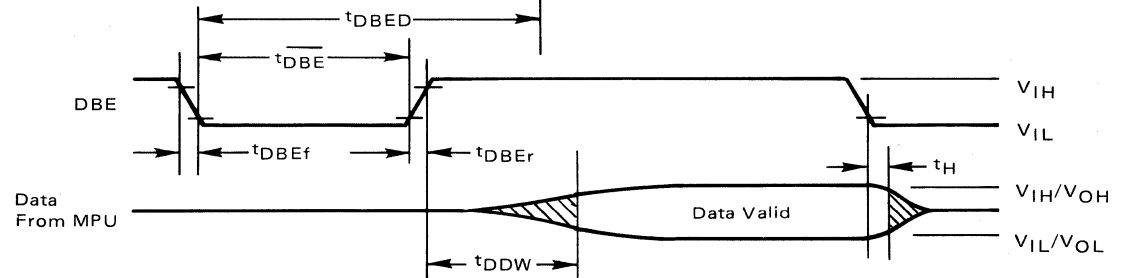
Read Timing



Write Timing, DBE =  $\phi 2$  Timing



Write Timing, DBE  $\neq \phi 2$  Timing



\*Output levels of 0.4 V and 2.4 V with input levels of 0.8 V and 2.0 V guarantee a minimum 0.4 V dynamic noise immunity at "1" and "0" levels.

 Data Not Valid

**TABLE 4 — DYNAMIC CHARACTERISTICS** ( $V_{CC} = 5.0 \text{ V} \pm 5\%$ ,  $V_{SS} = 0$ ,  $T_A = 0$  to  $70^\circ\text{C}$  unless otherwise noted.)

| Characteristic                                                            | Symbol                   | Min  | Typ | Max  | Unit          |
|---------------------------------------------------------------------------|--------------------------|------|-----|------|---------------|
| Address Setup Time                                                        | $t_{AS}$                 | 190  | —   | —    | ns            |
| Allowable Data Delay Time (Read)                                          | $t_{DDR}$                | —    | 200 | 350  | ns            |
| Memory Read Access Time                                                   | $t_{acc}$                | —    | —   | 540  | ns            |
| Data Setup Time (Read)                                                    | $t_{DSR}$                | 100  | —   | —    | ns            |
| Available Data Setup Time (Write)                                         | $t_{DSW}$                | 225  | —   | —    | ns            |
| Input Data Hold Time                                                      | $t_H$                    | 10   | —   | —    | ns            |
| Output Data Hold Time                                                     | $t_H$                    | 10   | 30  | —    | ns            |
| Address Hold Time                                                         | $t_{AH}$                 | 50   | 75  | —    | ns            |
| Address Delay                                                             | $t_{AD}$                 | —    | 220 | 300  | ns            |
| Data Delay (Write)                                                        | $t_{DDW}$                | —    | 165 | 225  | ns            |
| Interrupt Request Release Time                                            | $t_{IR}$                 | —    | 0.7 | 1.2  | $\mu\text{s}$ |
| Frequency of Operation                                                    | $f$                      | 0.1  | —   | 1.0  | MHz           |
| Clock Timing ( $\phi 1$ and $\phi 2$ )                                    |                          |      |     |      |               |
| Cycle Time                                                                | $t_{cyc}$                | 1.0  | —   | 10   | $\mu\text{s}$ |
| Clock Pulse Width                                                         | $PW_{\phi H}$            |      |     |      |               |
| (Measured at $V_{CC} - 0.3 \text{ V}$ ) $\phi 1$                          |                          | 430  | —   | 4500 | ns            |
| $\phi 2$                                                                  |                          | 450  | —   | 4500 | ns            |
| Clock Up Time                                                             | $t_{ut}$                 | 940  | —   | —    | ns            |
| Rise and Fall Times $\phi 1, \phi 2$                                      | $t_{\phi r}, t_{\phi f}$ | 5.0  | —   | 50   | ns            |
| (Measured between $V_{SS} + 0.3 \text{ V}$ and $V_{CC} - 0.3 \text{ V}$ ) |                          |      |     |      |               |
| Delay Time or Clock Separation                                            | $t_d$                    | 0    | —   | 9100 | ns            |
| (Measured at $V_{SS} + 0.5 \text{ V}$ )                                   |                          |      |     |      |               |
| Overshoot Duration                                                        | $t_{OS}$                 | 0    | —   | 40   | ns            |
| E Enable Pulse Width                                                      | $PW_E$                   | 0.45 | —   | 25   | $\mu\text{s}$ |
| E Enable Rise and Fall Time                                               | $t_{Er}, t_{Ef}$         | —    | —   | 25   | ns            |
| Processor Controls                                                        |                          |      |     |      |               |
| Processor Control Setup Time (Figure 12, 13, 14, 15)                      | $t_{PCS}$                | 200  | —   | —    | ns            |
| Processor Control Rise and Fall Time (Figures 12, 13, 14, 15)             | $t_{PCr}, t_{PCf}$       | —    | —   | 100  | ns            |
| Bus Available Delay (Figure 15)                                           | $t_{BA}$                 | —    | —   | 300  | ns            |
| Three State Enable (Figure 16)                                            | $t_{TSE}$                | —    | —   | 40   | ns            |
| Three State Delay (Figure 16)                                             | $t_{TSD}$                | —    | —   | 700  | ns            |
| Data Bus Enable Down Time During $\phi 1$ Up Time (Figure 6)              | $t_{DBE}$                | 150  | —   | —    | ns            |
| Data Bus Enable Delay (Figure 6)                                          | $t_{DBED}$               | 300  | —   | —    | ns            |
| Data Bus Enable Rise and Fall Times (Figure 6)                            | $t_{DBEr}, t_{DBEf}$     | —    | —   | 25   | ns            |

Note: Dynamic properties of most of the M6800 peripheral and memory parts exceed the requirements specified above, allowing for system flexibility.

Notice the logic levels specified in Figure 6. All parts are tested to output levels of 0.4 and 2.4 V with input levels of 0.8 and 2.0 V at the specified times. This will guarantee a minimum of 0.4 V dynamic noise immunity on the “1” and “0” levels.

#### Microprocessor Clocks $\phi 1$ and $\phi 2$

Figure 7 shows the microprocessor clocks, and Tables 3 and 4 show the static and dynamic clock specifications. The high level is specified at  $V_{IH}$  and the low level is specified at  $V_{IL}$ . The allowable clock frequency is specified by  $f$  (frequency). The clock overshoot is specified by  $V_{OS}$  (overshoot voltage) for a maximum duration of  $t_{OS}$  (overshoot time). The minimum  $\phi 1$  and  $\phi 2$  high level pulse widths are specified by  $PW_{\phi H}$  (pulse width high time). To guarantee the required access time for the peripherals, the clock up time,  $t_{ut}$ , is specified. Clock separation,  $t_d$ , is measured at a maximum voltage of  $V_{OV}$  (overlap voltage). This allows for a multitude of clock variations at the system frequency rate.

#### Maximum Ratings

Table 5 shows the maximum ratings. The most important specifications in the table from a user's standpoint are the operating power supply range of +4.75 V to +5.25 V and the operating temperature range of  $0^\circ\text{C}$  to  $70^\circ\text{C}$  ambient.

#### Test Conditions

The dynamic test load for the Data Bus, as shown in Figure 8, is 130 pF and one standard TTL load. The 11.7 k $\Omega$  resistor accounts for 205  $\mu\text{A}$  of 2.4 V level leakage current. The Address, R/W and VMA processor outputs have a dynamic test load of 90 pF and one TTL load as shown in Figure 8. The 16.5 k $\Omega$  resistor accounts for 145  $\mu\text{A}$  of 2.4 V level leakage current. The Interrupt Request test load is also shown in Figure 8. The load consists of a 3 k $\Omega$  resistor to  $V_{CC}$  and 100 pF of capacitance. Notice that the Data Bus lines, the Address lines, the Interrupt Request line and the E enable line are all specified and tested to guarantee 0.4 V of dynamic noise immunity on the “1” and “0” logic levels.

FIGURE 7 — MICROPROCESSOR  $\phi 1$  AND  $\phi 2$  CLOCKS

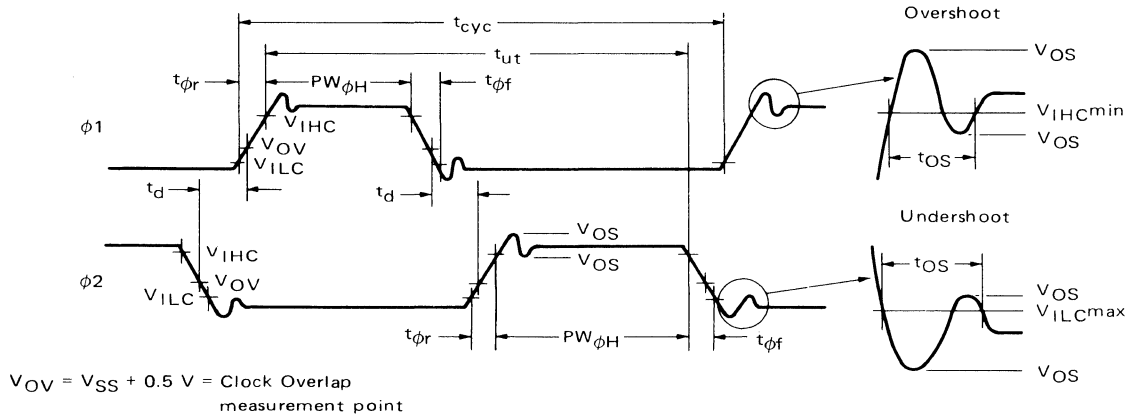
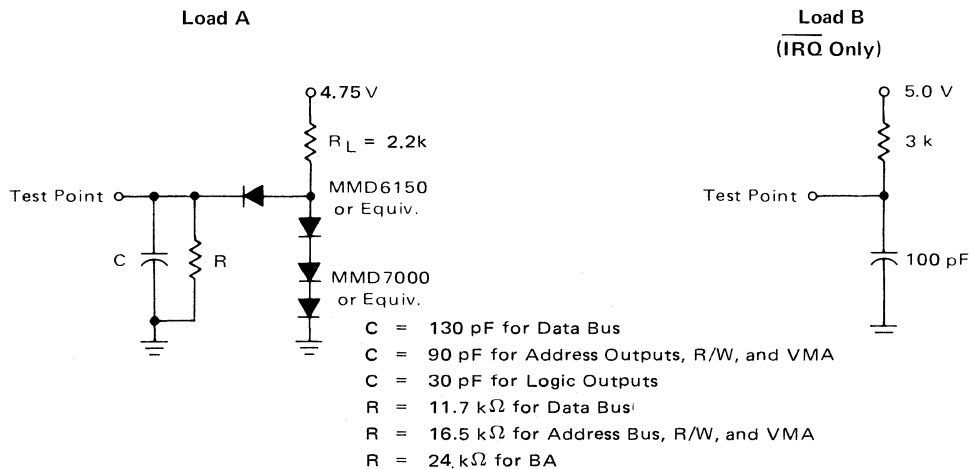


TABLE 5 — MAXIMUM RATINGS

| Ratings                     | Symbol    | Value        | Unit |
|-----------------------------|-----------|--------------|------|
| Supply Voltage              | $V_{CC}$  | -0.3 to +7.0 | Vdc  |
| Input Voltage               | $V_{in}$  | -0.3 to +7.0 | Vdc  |
| Operating Temperature Range | $T_A$     | 0 to +70     | °C   |
| Storage Temperature Range   | $T_{stg}$ | -55 to +150  | °C   |

FIGURE 8 — TEST LOADS



## A MINIMUM SYSTEM

A minimum M6800 system is defined as any size system within the basic load limitation of the MPU. Figure 9 shows a minimum system consisting of the microprocessor, three RAMs, two ROMs, two PIAs, and one ACIA. A maximum of eight peripherals are allowed on the bus in this configuration. This limitation is due to the capacitive loading specification of 130 pF on the microprocessor and peripheral data bus buffers — that is 30 pF of interconnect capacitance and 100 pF of data bus buffer capacitance.

Systems having more than eight peripherals can be implemented by using the Bus Extender (BEX), additional TTL gates and the microprocessor control lines. A useful curve showing the Data Bus and Address Bus drive capability with respect to Bus loading for typical conditions is shown in Figure 10. Data Bus buffer and Address Bus

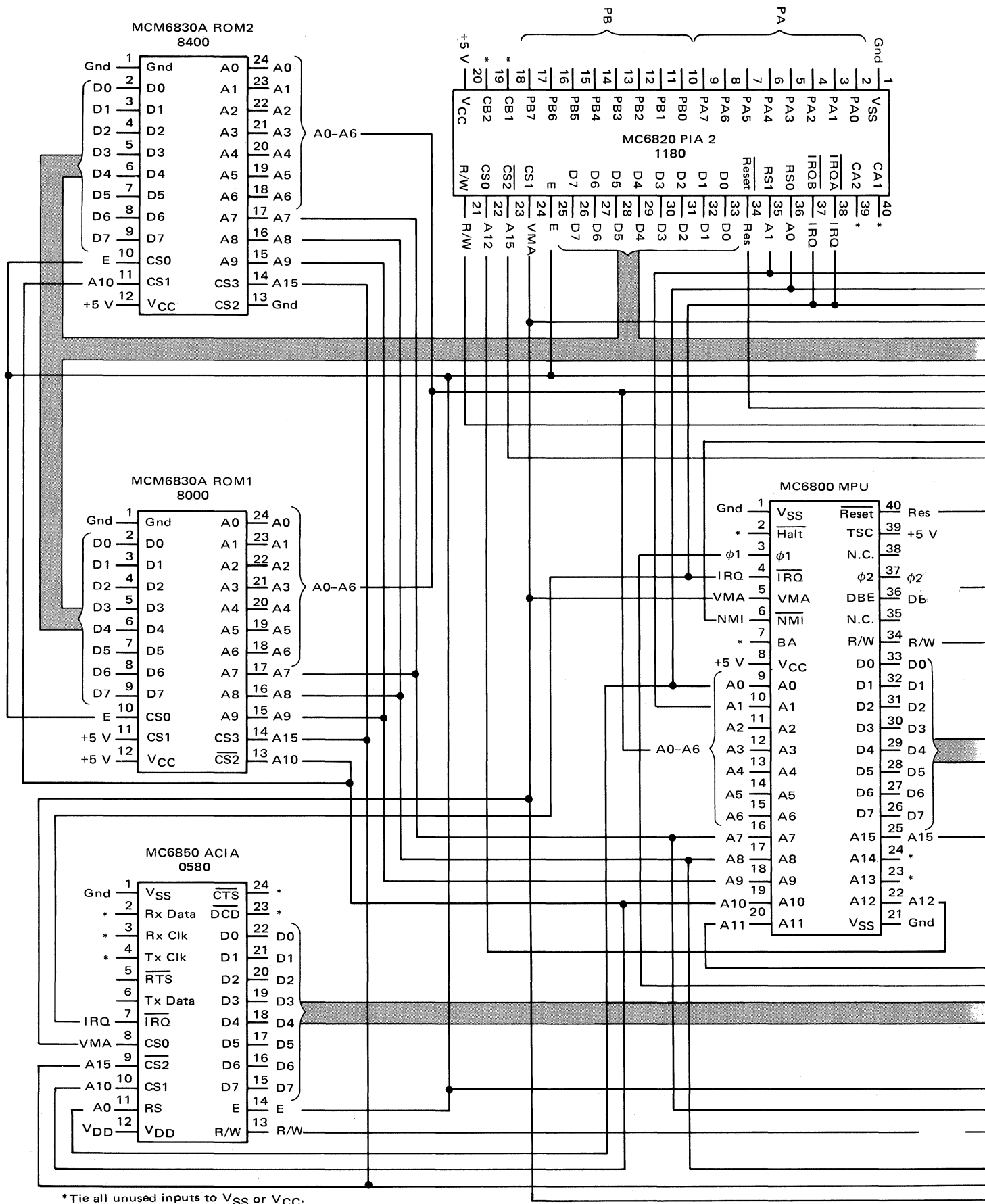
buffer delays typically increase at a rate of 0.5 ns/pF for pure capacitive loading.

## System Clock

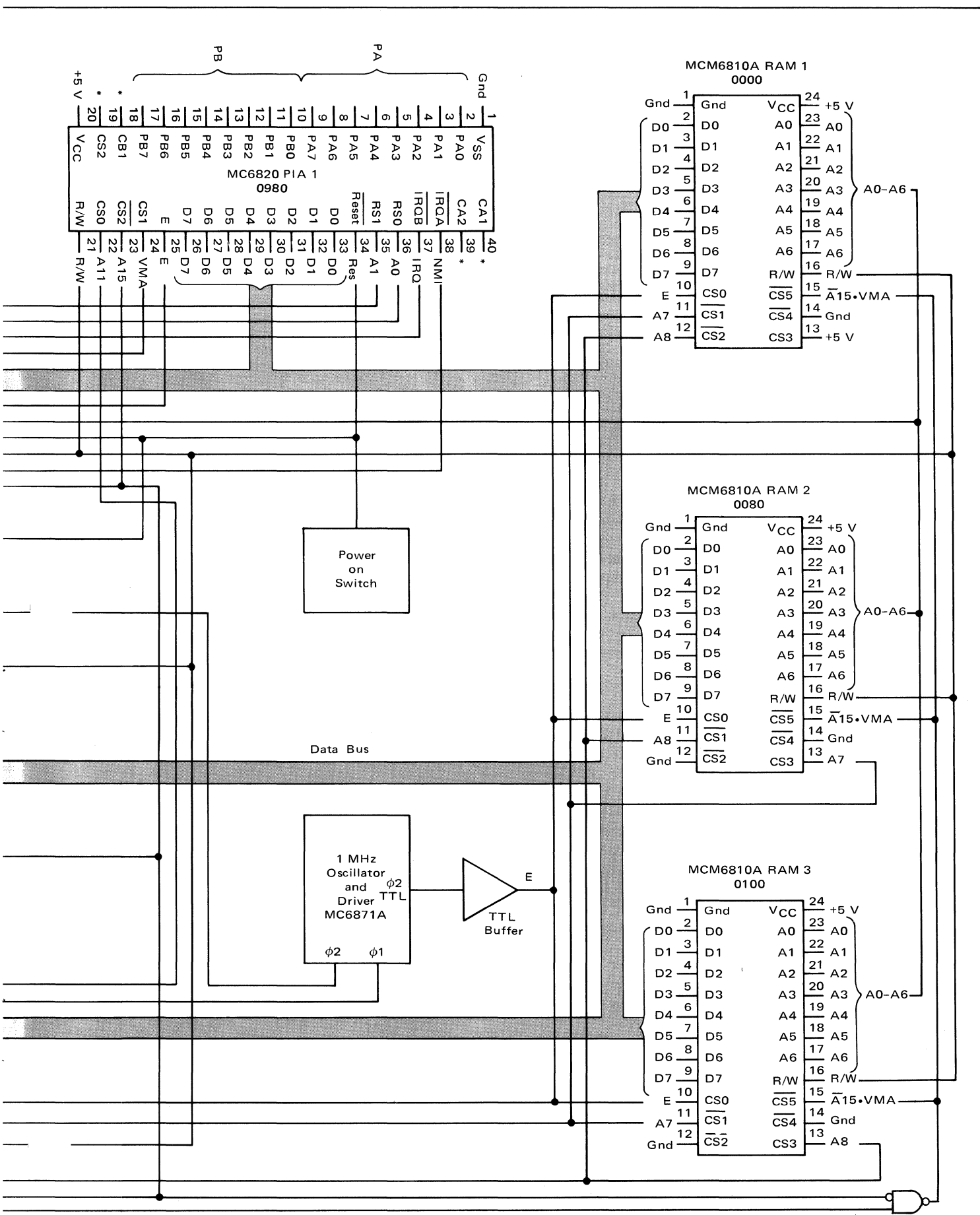
The microprocessor requires two clocks,  $\phi 1$  and  $\phi 2$ , as shown in Figure 9. A third clock, the E enable, which is in phase with  $\phi 2$ , is needed to transfer data to peripherals. Data transfers to the processor are made during the  $\phi 2$  time. The  $\phi 2$  clock is also tied to the microprocessor Data Bus Enable (DBE) which enables the Data Bus output buffers.

The write timing for this condition is shown in Figure 6 (DBE =  $\phi 2$ ) and Table 4. If additional data setup or hold time is required on an MPU write, the DBE down-time can be decreased as shown in Figure 6 (DBE  $\neq \phi 2$ ). The minimum down-time for DBE is  $t_{DBE}$  as shown in Table 4,

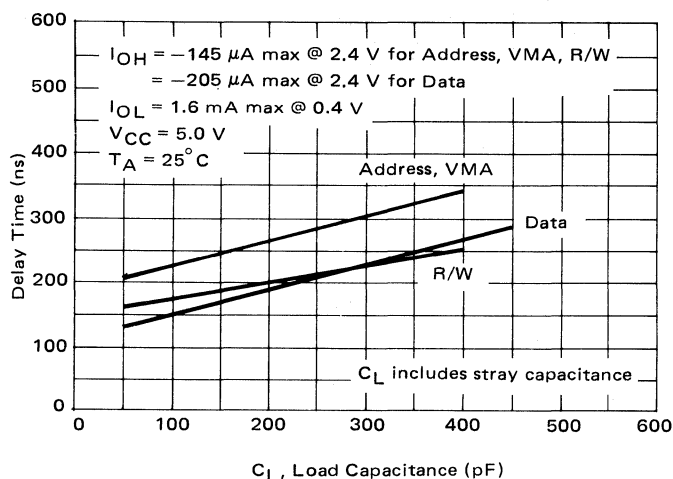
FIGURE 9 – TYPICAL SYSTEM INTERCONNECTION







**FIGURE 10 — TYPICAL BUS DELAY TIMES versus CAPACITIVE LOADING**



and must occur within  $\phi 1$  up-time. The minimum delay from the trailing edge of DBE to the trailing edge of  $\phi 1$  is  $t_{DBED}$ . By skewing DBE with respect to E in this manner, data setup or hold time can be increased.

The capacitive drive requirement for the E enable line for this system is 90 pF — 60 pF for peripheral E input capacitance and 30 pF for interconnections. The MC6871A clock circuit provides  $\phi 1$  and  $\phi 2$  signals which meet the MPU requirements. It also provides a TTL compatible E enable output ( $\phi 2$ , TTL) which leads  $\phi 2$  by approximately one TTL driver circuit stage delay. In the system shown, the Data Bus buffers go to their three-state mode with the trailing edge of  $\phi 2$  and E allowing for sufficient hold time ( $\approx 300$  ns). In systems having TTL loads on the Data Bus, E can be tied to DBE. Variations between E and  $\phi 2$  do not affect the processor or peripheral Data Bus hold time requirements providing the trailing edge of E occurs after the trailing edge of  $\phi 2$ .

### Addressing Peripherals and Memory

As shown in Figure 6, all Address lines are valid by address setup time  $t_{AS}$ . The Address lines, Chip Selects and the E enable are tied to the ROM as shown in Figure 9. The Address lines required for addressing the ROMs are shown in Table 6. A true Address bit selects a true Chip Select, and an Address bit selects a Chip Select. Figure 9 also shows the Address, Chip Select, E enable and R/W interconnection for the RAMs, the PIAs, and the ACIA, with Table 6 showing the Address lines required.

Notice that VMA is used as a Chip Select to the peripherals. VMA is needed because interrupts on these parts are cleared on a read of data from the peripherals. The microprocessor requires from 2 to 12 cycles to do an instruction. During those portions of an instruction in which the Data Bus is not active, the R/W line is held high, the Address lines are in an indeterminate state and the Data Bus buffers are in an indeterminate state. Thus a false read of a peripheral could occur, and if that peripheral was interrupting the microprocessor, the interrupt would be cleared. Tying VMA into one of the Chip Selects solves this problem since VMA is high only when there is a valid memory address on the Address Bus. If the  $\overline{\text{Halt}}$  control line, the

Wait for Interrupt (WAI) instruction, or the TSC control line is used, Address and R/W lines float, which could result in a false write into memory. If  $\overline{\text{Halt}}$ , TSC and WAI are required in a minimum system, a TTL AND gate is used to AND one of the chip selects with VMA as shown.

Address space was chosen as shown in Table 6 for the following reasons: When the microprocessor power comes up, the address FFFE goes out on the Address lines, followed by FFFF (the address of the start up program is stored at FFFE and FFFF). Thus, A15 was chosen to select the ROMs. The ROM Address bits are A0 thru A9. A10 was chosen as the other Chip Select so that ROM addressing would be contiguous. The ROM Chip Selects are programmable; for this example CS1 and CS3 are defined as true and CS2 as not true (Chip Select).

A15 can be used to select all peripherals except the ROMs. The RAM Address bits are A0 thru A6. A7 and A8 make the address space for the RAMs contiguous.

When using direct addressing, the Address Bus high bits (A8 thru A15) are all zeros. Then RAM 1 and RAM 2 in Table 6 form 256 contiguous locations for direct addressing. When selecting the ACIA and PIAs, the RAMs are deselected by keeping A7 and A8 high.

The ACIA is selected with  $\overline{\text{A15}}$ , VMA, and A10. A10 can be reused since the state of A15 determines whether A10 is used as a ROM Address line or the ACIA Address line. The PIAs are selected with  $\overline{\text{A15}}$ , VMA and two of the remaining unused address lines — A11 and A12.

**TABLE 6 — ADDRESS LINES FOR SYSTEM OF FIGURE 9**

| Device | Address | Chip Selects                                                              | Select Code |
|--------|---------|---------------------------------------------------------------------------|-------------|
| ROM 1  | A0-A9   | $\overline{\text{A10}}$ A15                                               | 8000        |
| ROM 2  | A0-A9   | A10 A15                                                                   | 8400        |
| RAM 1  | A0-A6   | $\overline{\text{A7}}$ $\overline{\text{A8}}$ $\overline{\text{A15}}$ VMA | 0000        |
| RAM 2  | A0-A6   | A7 $\overline{\text{A8}}$ $\overline{\text{A15}}$ VMA                     | 0080        |
| RAM 3  | A0-A6   | $\overline{\text{A7}}$ A8 $\overline{\text{A15}}$ VMA                     | 0100        |
| ACIA 1 | A0      | A10 VMA $\overline{\text{A15}}$ *                                         | 0580        |
| PIA 1  | A0 A1   | A11 VMA $\overline{\text{A15}}$ *                                         | 0980        |
| PIA 2  | A0 A1   | A12 VMA $\overline{\text{A15}}$ *                                         | 1180        |

\*When addressing the ACIA and PIAs, A7 and A8 must be high to disable the RAMs. When addressing the RAMs, A10, A11 and A12 are held low.

### MPU ADDRESS MODES

The MC6800 has a set of 72 different instructions. Included are binary and decimal arithmetic, logical, shift, rotate, load, store, conditional or unconditional branch, interrupt and stack manipulation instructions.

Table 7 shows the microprocessor instruction set. Table 8 shows the instruction addressing modes and associated execution times.

The MPU address modes are:

1. Immediate
2. Direct
3. Indexed
4. Extended
5. Implied
6. Relative
7. Accumulator (ACCX)

TABLE 7 – MICROPROCESSOR INSTRUCTION SET – ALPHABETIC SEQUENCE

|     |                                 |     |                          |     |                                              |
|-----|---------------------------------|-----|--------------------------|-----|----------------------------------------------|
| ABA | Add Accumulators                | CLR | Clear                    | PUL | Pull Data                                    |
| ADC | Add with Carry                  | CLV | Clear Overflow           | ROL | Rotate Left                                  |
| ADD | Add                             | CMP | Compare                  | ROR | Rotate Right                                 |
| AND | Logical And                     | COM | Complement               | RTI | Return from Interrupt                        |
| ASL | Arithmetic Shift Left           | CPX | Compare Index Register   | RTS | Return from Subroutine                       |
| ASR | Arithmetic Shift Right          | DAA | Decimal Adjust           | SBA | Subtract Accumulators                        |
| BCC | Branch if Carry Clear           | DEC | Decrement                | SBC | Subtract with Carry                          |
| BCS | Branch if Carry Set             | DES | Decrement Stack Pointer  | SEC | Set Carry                                    |
| BEQ | Branch if Equal to Zero         | DEX | Decrement Index Register | SEI | Set Interrupt Mask                           |
| BGE | Branch if Greater or Equal Zero | EOR | Exclusive OR             | SEV | Set Overflow                                 |
| BGT | Branch if Greater than Zero     | INC | Increment                | STA | Store Accumulator                            |
| BHI | Branch if Higher                | INS | Increment Stack Pointer  | STS | Store Stack Register                         |
| BIT | Bit Test                        | INX | Increment Index Register | STX | Store Index Register                         |
| BLE | Branch if Less or Equal         | JMP | Jump                     | SUB | Subtract                                     |
| BLS | Branch if Lower or Same         | JSR | Jump to Subroutine       | SWI | Software Interrupt                           |
| BLT | Branch if Less than Zero        | LDA | Load Accumulator         | TAB | Transfer Accumulators                        |
| BMI | Branch if Minus                 | LDS | Load Stack Pointer       | TAP | Transfer Accumulators to Condition Code Reg. |
| BNE | Branch if Not Equal to Zero     | LDX | Load Index Register      | TBA | Transfer Accumulators                        |
| BPL | Branch if Plus                  | LSR | Logical Shift Right      | TPA | Transfer Condition Code Reg. to Accumulator  |
| BRA | Branch Always                   | NEG | Negate                   | TST | Test                                         |
| BSR | Branch to Subroutine            | NOP | No Operation             | TSX | Transfer Stack Pointer to Index Register     |
| BVC | Branch if Overflow Clear        | ORA | Inclusive OR Accumulator | TXS | Transfer Index Register to Stack Pointer     |
| BVS | Branch if Overflow Set          | PSH | Push Data                | WAI | Wait for Interrupt                           |
| CBA | Compare Accumulators            |     |                          |     |                                              |
| CLC | Clear Carry                     |     |                          |     |                                              |
| CLI | Clear Interrupt Mask            |     |                          |     |                                              |

A description of these address modes follows, with emphasis on what is occurring in each machine cycle on the Address Bus, Data Bus, R/W line and the VMA line. A cycle-by-cycle representation is shown in Figure 11. When VMA is low the Data Bus and Address Bus are in an indeterminate state. (See MC6800 data sheet for a cycle-by-cycle description of the entire instruction set.)

### Immediate Addressing

Immediate addressing is an addressing technique in which the first byte of the instruction contains the operator and the second byte contains the operand. Exceptions to this are the LDS and LDX instructions which have the operand in the second and third bytes of the instruction.

Referring to Figure 11, during the first half of cycle #1 the Program Counter (PC) current address is put on the line, R/W goes high designating a read operation and VMA goes high designating the current address is a valid memory address. During the second half of cycle #1, the LDA operator code is put on the Data Bus from memory and loaded into the MPU. The MPU Program Counter is incremented and the operand, which is usually data, is loaded into the MPU on cycle #2. The operator of the next instruction follows on the next cycle.

### Direct Addressing

In direct addressing, the address of the operand is contained in the second byte of the instruction. Direct addressing allows the user to directly address the lowest 256 bytes in the machine, i.e., locations zero through 255. Enhanced execution times are achieved by storing data in these locations. In most configurations, it should be a Random Access Memory.

Referring to Figure 11, the address in the Program Counter is put on the address bus and the LDA operator code is loaded into the MPU during cycle #1. The Program Counter is incremented and during cycle #2, 8 bits of

address are loaded into the Address Bus Low (ABL) register which contains the lower 8 bits of the 16-bit address register of the MPU. The upper 8 bits, which are contained in the Address Bus High (ABH) register, are forced to all zeros. In the third cycle the new address is put on the Address Bus and the operand is loaded into the MPU.

A STA instruction is handled in the same manner as an LDA instruction except there is an additional cycle required, which is cycle #6 in the diagram. Due to the MPU architecture this additional cycle is required to move the accumulator internally in the machine. During this cycle the Data Bus is in an indeterminate state and VMA goes low. The actual storing of data then occurs on cycle #7. The next instruction follows as shown.

### Indexed Addressing

In indexed addressing, the address contained in the second byte of the instruction is added to the index register's lowest 8 bits in the MPU. The carry is then added to the higher order 8 bits of the index register. This result is then used to address memory. The modified address is held in a temporary address register so there is no change to the index register. These are two byte instructions.

In the first two cycles of an indexed address instruction, the LDA operator code is loaded into the MPU followed by the index offset. In the third cycle the low order byte of the index register is loaded into the adder and added to the offset; the carry propagates during cycle #4. The VMA goes low during these two cycles while the MPU is preparing the indexed address. The new address then goes on the line during cycle #5.

The STA indexed instruction is handled in the same manner as the LDA instruction, again with the exception that an additional cycle is required due to the MPU architecture. VMA is held low then for three cycles for the STA instruction instead of two for the LDA instruction.

**TABLE 8 – INSTRUCTION ADDRESSING MODES AND ASSOCIATED EXECUTION TIMES**  
(Times in Machine Cycles)

|     | (Dual Operand) | ACCX | Immediate | Direct | Extended | Indexed | Implied | Relative |     | (Dual Operand) | ACCX | Immediate | Direct | Extended | Indexed | Implied |
|-----|----------------|------|-----------|--------|----------|---------|---------|----------|-----|----------------|------|-----------|--------|----------|---------|---------|
| ABA |                | •    | •         | •      | •        | •       | 2       | •        | INC |                | 2    | •         | •      | 6        | 7       | •       |
| ADC | x              | •    | 2         | 3      | 4        | 5       | •       | •        | INS |                | •    | •         | •      | •        | •       | 4       |
| ADD | x              | •    | 2         | 3      | 4        | 5       | •       | •        | INX |                | •    | •         | •      | •        | •       | 4       |
| AND | x              | •    | 2         | 3      | 4        | 5       | •       | •        | JMP |                | •    | •         | •      | 3        | 4       | •       |
| ASL |                | 2    | •         | •      | 6        | 7       | •       | •        | JSR |                | •    | •         | •      | 9        | 8       | •       |
| ASR |                | 2    | •         | •      | 6        | 7       | •       | •        | LDA | x              | •    | 2         | 3      | 4        | 5       | •       |
| BCC |                | •    | •         | •      | •        | •       | •       | 4        | LDS |                | •    | 3         | 4      | 5        | 6       | •       |
| BCS |                | •    | •         | •      | •        | •       | •       | 4        | LDX |                | •    | 3         | 4      | 5        | 6       | •       |
| BEA |                | •    | •         | •      | •        | •       | •       | 4        | LSR |                | 2    | •         | •      | 6        | 7       | •       |
| BGE |                | •    | •         | •      | •        | •       | •       | 4        | NEG |                | 2    | •         | •      | 6        | 7       | •       |
| BGT |                | •    | •         | •      | •        | •       | •       | 4        | NOP |                | •    | •         | •      | •        | •       | 2       |
| BHI |                | •    | •         | •      | •        | •       | •       | 4        | ORA | x              | •    | 2         | 3      | 4        | 5       | •       |
| BIT | x              | •    | 2         | 3      | 4        | 5       | •       | •        | PSH |                | •    | •         | •      | •        | •       | 4       |
| BLE |                | •    | •         | •      | •        | •       | •       | 4        | PUL |                | •    | •         | •      | •        | •       | 4       |
| BLS |                | •    | •         | •      | •        | •       | •       | 4        | ROL |                | 2    | •         | •      | 6        | 7       | •       |
| BLT |                | •    | •         | •      | •        | •       | •       | 4        | ROR |                | 2    | •         | •      | 6        | 7       | •       |
| BMI |                | •    | •         | •      | •        | •       | •       | 4        | RTI |                | •    | •         | •      | •        | •       | 10      |
| BNE |                | •    | •         | •      | •        | •       | •       | 4        | RTS |                | •    | •         | •      | •        | •       | 5       |
| BPL |                | •    | •         | •      | •        | •       | •       | 4        | SBA |                | •    | •         | •      | •        | •       | 2       |
| BRA |                | •    | •         | •      | •        | •       | •       | 4        | SBC | x              | •    | 2         | 3      | 4        | 5       | •       |
| BSR |                | •    | •         | •      | •        | •       | •       | 8        | SEC |                | •    | •         | •      | •        | •       | 2       |
| BVC |                | •    | •         | •      | •        | •       | •       | 4        | SEI |                | •    | •         | •      | •        | •       | 2       |
| BVS |                | •    | •         | •      | •        | •       | •       | 4        | SEV |                | •    | •         | •      | •        | •       | 2       |
| CBA |                | •    | •         | •      | •        | •       | 2       | •        | STA | x              | •    | •         | 4      | 5        | 6       | •       |
| CLC |                | •    | •         | •      | •        | •       | 2       | •        | STS |                | •    | •         | 5      | 6        | 7       | •       |
| CLI |                | •    | •         | •      | •        | •       | 2       | •        | STX |                | •    | •         | 5      | 6        | 7       | •       |
| CLR |                | 2    | •         | •      | 6        | 7       | •       | •        | SUB | x              | •    | 2         | 3      | 4        | 5       | •       |
| CLV |                | •    | •         | •      | •        | •       | 2       | •        | SWI |                | •    | •         | •      | •        | •       | 12      |
| CMP | x              | •    | 2         | 3      | 4        | 5       | •       | •        | TAB |                | •    | •         | •      | •        | •       | 2       |
| COM |                | 2    | •         | •      | 6        | 7       | •       | •        | TAP |                | •    | •         | •      | •        | •       | 2       |
| CPX |                | •    | 3         | 4      | 5        | 6       | •       | •        | TBA |                | •    | •         | •      | •        | •       | 2       |
| DAA |                | •    | •         | •      | •        | •       | 2       | •        | TPA |                | •    | •         | •      | •        | •       | 2       |
| DEC |                | 2    | •         | •      | 6        | 7       | •       | •        | TST |                | 2    | •         | •      | 6        | 7       | •       |
| DES |                | •    | •         | •      | •        | •       | 4       | •        | TSX |                | •    | •         | •      | •        | •       | 4       |
| DEX |                | •    | •         | •      | •        | •       | 4       | •        | TSX |                | •    | •         | •      | •        | •       | 4       |
| EOR | x              | •    | 2         | 3      | 4        | 5       | •       | •        | WAI |                | •    | •         | •      | •        | •       | 9       |

Note: Interrupt time is 12 cycles from the end of the instruction being executed, except following a WAI instruction. Then it is 4 cycles.

### Extended Addressing

In extended addressing, the address contained in the second byte of the instruction is the 8 higher order address bits of the operand. The third byte of the instruction contains the lower 8 bits of the address. This is an absolute address in memory and these are 3 byte instructions. Extended addressing is the same as direct addressing except the address field is a full 16 bits. This means the LDA and STA instructions require an additional cycle (cycles #2 and #6) to fetch the high order 8 bits of the address. Notice that in cycle #8 VMA is low signifying the Data Bus is in an indeterminate state. This is again due to the internal architecture of the MPU.

### Implied Addressing

There are two types of implied addressing instructions: those which do not require an address and those which do require an address. These are one byte instructions. An

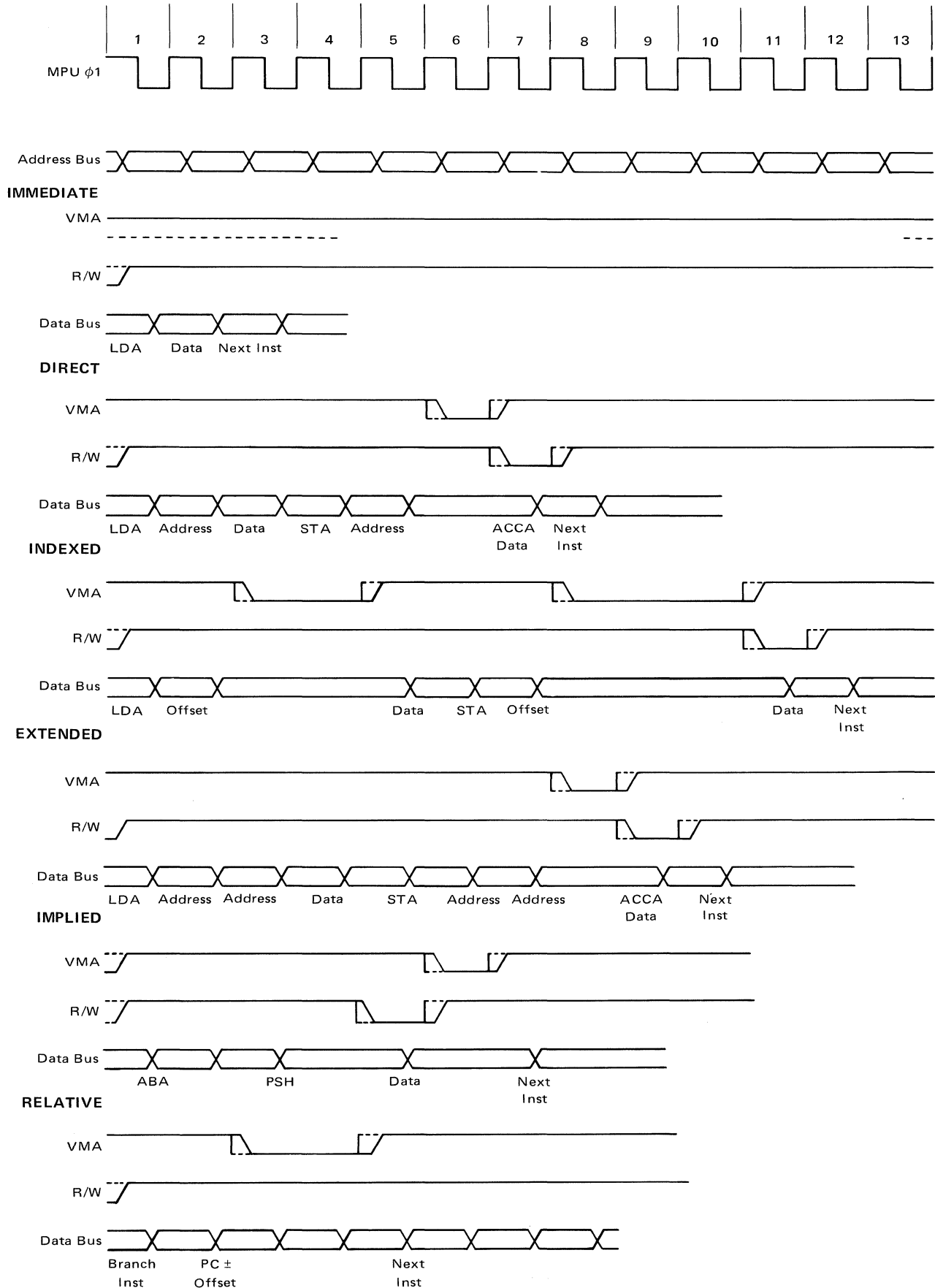
example of an instruction which does not require an address is ABA. Here the contents of accumulator A is added to accumulator B and the result put in accumulator A. The Data Bus and Address Bus are valid only on the first cycle of the instruction as shown in Figure 11.

For those instructions which do require an address, the address is held by an internal MPU register such as the stack pointer. Thus no data is required to develop an address. An example of this type of instruction is PSH.

Looking again at Figure 11, the Data Bus and Address Bus are valid on cycle #3 to fetch the PSH operand. In cycle #4 the MPU is moving data internally and VMA does not go low. During this time the Address bus contains the address of the next instruction in ROM and the MPU is doing an invalid Read. In cycle #5 the stack pointer is loaded into Address Bus buffers and data is written into the stack. On the next cycle VMA goes low and the stack pointer is decremented. The next instruction then follows.



FIGURE 11 – ADDRESS MODES





## Relative Addressing

In relative addressing, the address contained in the second byte of the instruction is added to the program counter's lowest 8 bits plus two. The carry or borrow is then added to the high 8 bits. This allows the user to address data within a range of -126 to +129 bytes of the present instruction. These are two byte instructions which perform branch functions.

Referring again to Figure 11, the operator code is fetched from the memory location stored in the instruction register during cycle #1. The Program Counter offset is loaded into the MPU during cycle #2. The offset is added to the low order bits of the Program Counter in cycle #3 and the carry is propagated during cycle #4. During these two cycles VMA goes low while the MPU is operating on the offset data. The next instruction is loaded during cycle #5.

## Accumulator (ACCX) Addressing

In accumulator only addressing, either accumulator A or accumulator B is specified.

An example is ASLA (Arithmetic Shift Left, on the A accumulator). These are one byte, two cycle instructions and the Address Bus and Data Bus are active only during the first cycle of the instruction when the operator is being loaded from memory. During the second cycle the machine performs the operation and VMA is high.

## PROCESSOR CONTROLS

The microprocessor has six Processor Control Lines (Figure 1) which are:

|       |                          |
|-------|--------------------------|
| Reset |                          |
| NMI   | (Non-Maskable Interrupt) |
| Halt  |                          |
| BA    | (Bus Available)          |
| TSC   | (Three-State Control)    |
| DBE   | (Data Bus Enable)        |

Two of the control lines, Reset and DBE are required for all systems. The remaining control lines can be utilized to enhance throughput and flexibility depending on the system application. The simplicity of the processor control lines results directly from the simplicity of the M6800 system architecture.

Along with the discussion on NMI will be a discussion of the Wait For Interrupt (WAI) instruction and the Interrupt Request (IRQ) line since the three interrupt types are closely aligned.

### Reset

The Reset input is used to reset and start the MPU from a power down condition resulting from a power failure or initial start-up of the processor. This input can also be used to reinitialize the machine at any time after start up.

If a high level is detected in this input, this will signal the MPU to begin the reset sequence. During the reset sequence, the contents of the last two locations (FFFE, FFFF) in memory will be loaded into the Program Counter to point to the beginning of the reset routine. During the reset routine, the interrupt mask bit is set and must be cleared under program control before the MPU can be interrupted by IRQ. While Reset is low (assuming a minimum of 8 clock cycles have occurred) the MPU output signals will be in the following states: VMA = low, BA = low, Data Bus = high impedance, R/W = high (read state), and the Address Bus will contain the reset address FFFE. Figure 12 illustrates a power up sequence using the Reset control line. After the power supply reaches 4.75 V a minimum of eight clock cycles are required for the processor to stabilize in preparation for restarting. During these eight cycles, VMA will be in an indeterminate state so any devices that are enabled by VMA which could accept a false write during this time (such as a battery-backed RAM) must be disabled until VMA is forced low after eight cycles. Reset can go high asynchronously with the system clock any time after the eighth cycle.

Reset timing is shown in Figure 12 and Table 4. The maximum rise and fall transition times are specified by tPCr and tPCf. If Reset is high at tPCS (processor control setup time) as shown in Figure 12 in any given cycle, then the restart sequence will begin on the next cycle as shown. The Reset control line may also be used to reinitialize the MPU system at any time during its operation. This is accomplished by pulsing Reset low for the duration of a minimum of three complete  $\phi 2$  cycles. The Reset pulse can be completely asynchronous with the MPU system clock and will be recognized during  $\phi 2$  if setup time tPCS is met.

Reset timing is shown in Figure 12 and Table 4. The maximum rise and fall transition times are specified by tPCr and tPCf. If Reset is high at tPCS (processor control setup time) as shown in Figure 12 in any given cycle, then the restart sequence will begin on the next cycle as shown. The Reset control line may also be used to reinitialize the MPU system at any time during its operation. This is accomplished by pulsing Reset low for the duration of a minimum of three complete  $\phi 2$  cycles. The Reset pulse can be completely asynchronous with the MPU system clock and will be recognized during  $\phi 2$  if setup time tPCS is met.

## Non-Maskable Interrupt (NMI) and Wait for Interrupt (WAI)

The MC6800 is capable of handling two types of interrupts: maskable (IRQ) as described earlier, and non-maskable (NMI). IRQ is maskable by the interrupt mask in the condition code register while NMI is not maskable. The handling of these interrupts by the MPU is the same except that each has its own vector address. The behavior of the MPU when interrupted is shown in Figure 13, which details the MPU response to an interrupt while the MPU is executing the control program. The interrupt shown could be either IRQ or NMI and can be asynchronous with respect to  $\phi 2$ . The interrupt is shown going low at time tPCS in cycle #1 which precedes the first cycle of an instruction (OP code fetch). This instruction is not executed but instead the Program Counter (PC), Index Register (IX), Accumulators (ACCX), and the Condition Code Register (CCR) are pushed onto the stack.

The Interrupt Mask bit is set to prevent further interrupts. The address of the interrupt service routine is then fetched from FFFC, FFFD for an NMI interrupt and from FFF8, FFF9 for an IRQ interrupt. Upon completion of the interrupt service routine, the execution of RTI will pull the PC, IX, ACCX, and CCR off of the stack; the Interrupt Mask bit is restored to its condition prior to Interrupts.

Figure 14 is a similar interrupt sequence, except in this case, a WAIT instruction has been executed in preparation for the interrupt. This technique speeds up the MPU's response to the interrupt because the stacking of the PC, IX, ACCX, and the CCR is already done. While the MPU is waiting for the interrupt, Bus Available will go high indicating the following states of the control lines: VMA is low, and the Address Bus, R/W and Data Bus are all in the high impedance state. After the interrupt occurs, it is serviced as previously described.

## Halt and Single Instruction Execution

The Halt line provides an input to the MPU to allow control of program execution by an outside source. If Halt is high, the MPU will execute the instructions; if it is low, the MPU will go to a halted or idle mode. A response signal, Bus Available (BA) provides an indication of the current

FIGURE 12 – RESET TIMING

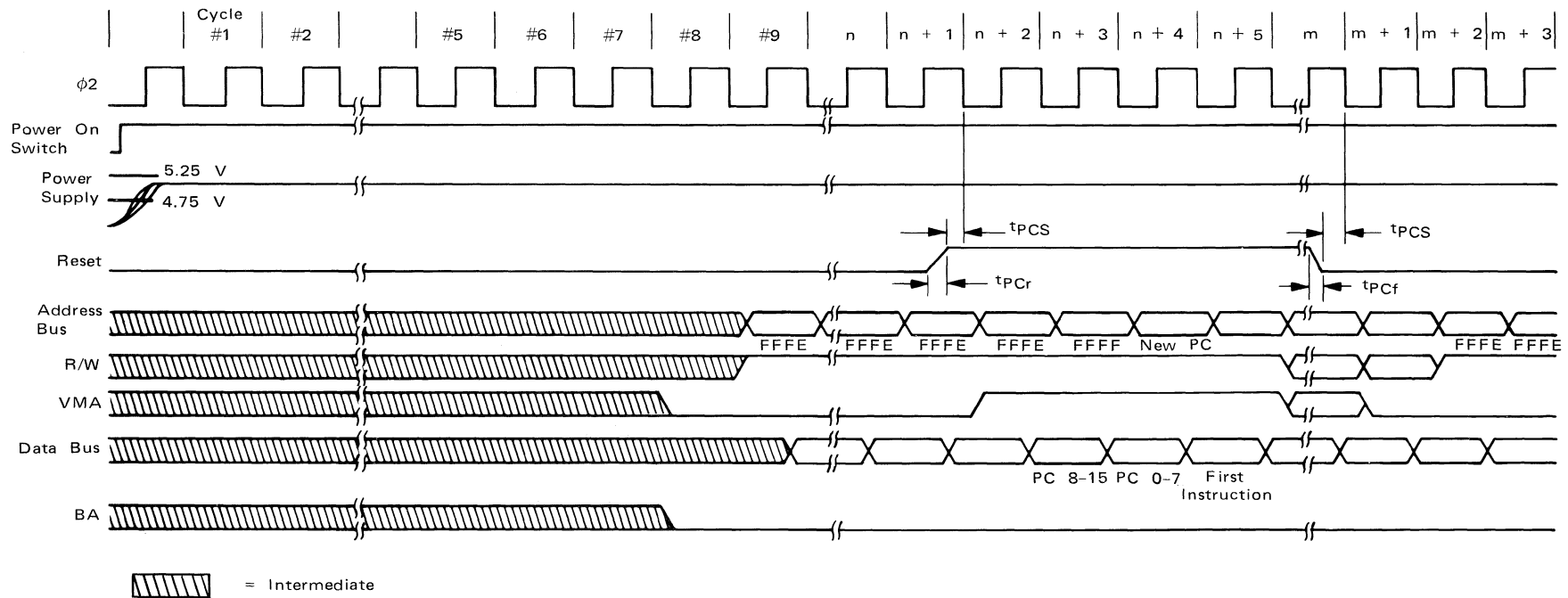


FIGURE 13 – INTERRUPT TIMING

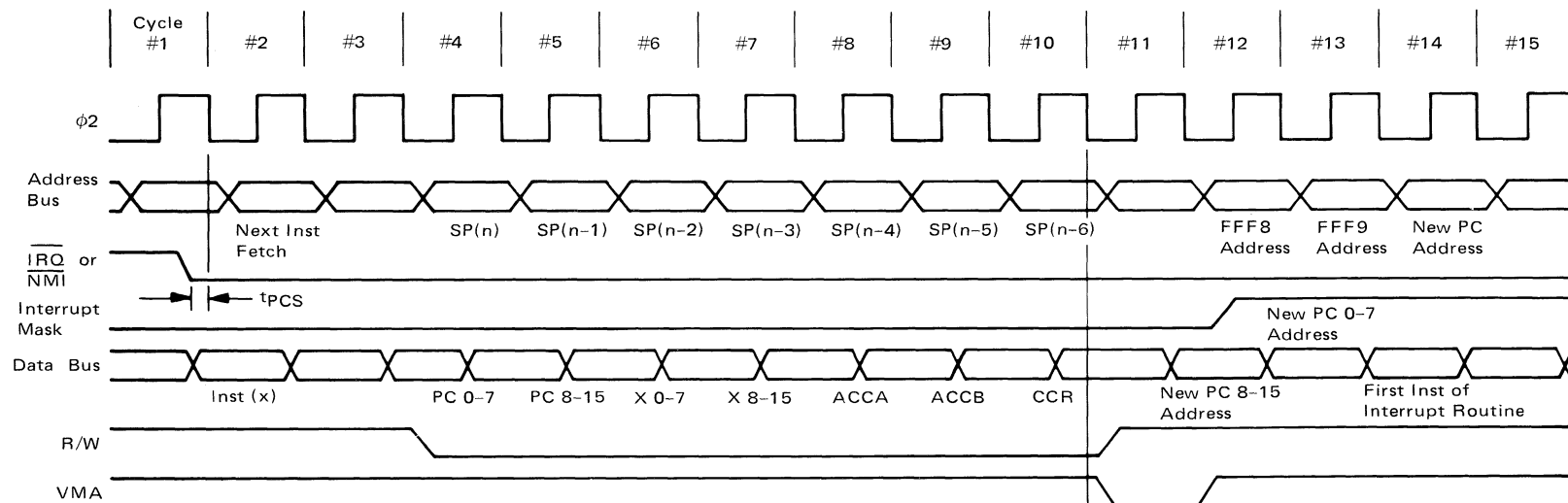
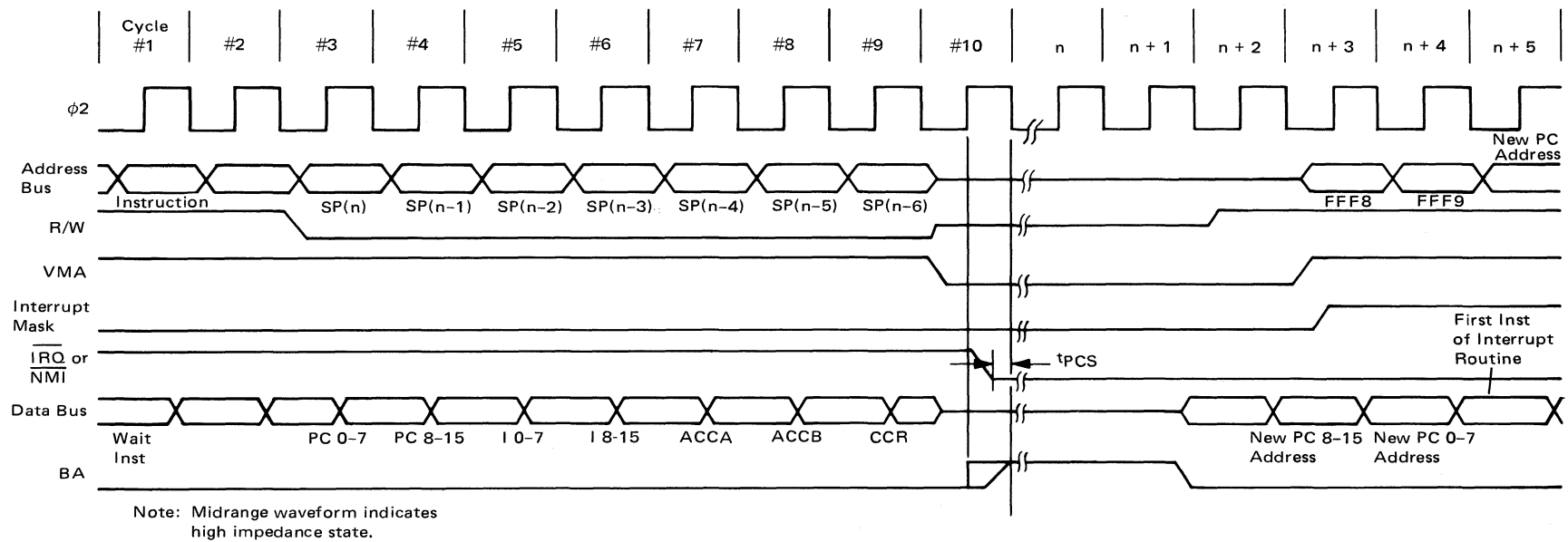


FIGURE 14 – WAIT INSTRUCTION TIMING



MPU status. When BA is low, the MPU is in the process of executing the control program; if BA is high, the MPU has halted and all internal activity has stopped.

When BA is high, the Address Bus, Data Bus, and R/W line will be in a high impedance state, effectively removing the MPU from the system bus. VMA is forced low so that the floating system bus will not activate any device on the bus that is enabled by VMA.

While the MPU is halted, all program activity is stopped, and if either an NMI or IRQ interrupt occurs, it will be latched into the MPU and acted on as soon as the MPU is taken out of the halted mode. If a Reset command occurs while the MPU is halted, the following states occur: VMA = low, BA = low, Data Bus = high impedance, R/W = high (read state), and the Address Bus will contain address FFFE as long as  $\overline{\text{Reset}}$  is low. As soon as the  $\overline{\text{Halt}}$  line goes high, the MPU will go to locations FFFE and FFFF for the address of the reset routine.

Figure 15 shows the timing relationships involved when halting the MPU. The instruction illustrated is a one byte, 2 cycle instruction such as CLRA. When  $\overline{\text{Halt}}$  goes low, the MPU will halt after completing execution of the current instruction. The transition of  $\overline{\text{Halt}}$  must occur tPCS before the trailing edge of  $\phi 1$  of the last cycle of an instruction (point A of Figure 15).  $\overline{\text{Halt}}$  must not go low any time later than the minimum tPCS specified.

The fetch of the OP code by the MPU is the first cycle of the instruction. If  $\overline{\text{Halt}}$  had not been low at Point A but went low during  $\phi 2$  of that cycle, the MPU would have halted after completion of the following instruction. BA will go high by time tBA (bus available delay time) after the last instruction cycle. At this point in time, VMA is low and R/W, Address Bus, and the Data Bus are in the high impedance state.

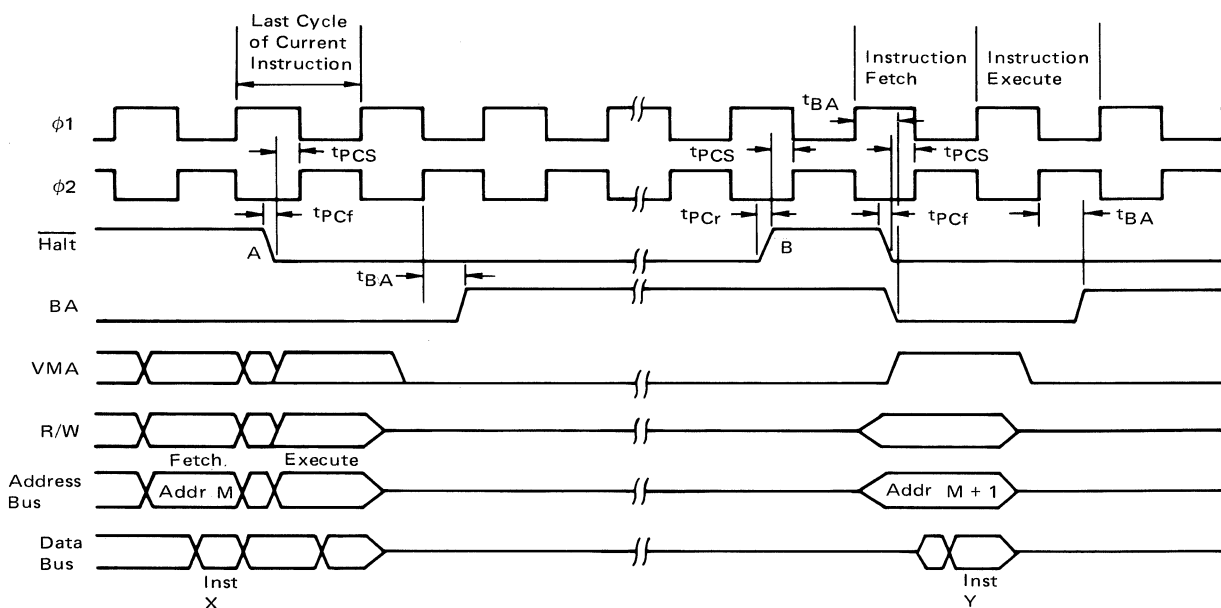
To debug programs it is advantageous to step through programs instruction by instruction. To do this,  $\overline{\text{Halt}}$  must be brought high for one MPU cycle and then returned low as shown at point B of Figure 15. Again, the transitions of  $\overline{\text{Halt}}$  must occur tPCS before the trailing edge of  $\phi 1$ . BA will go low at tBA after the leading edge of the next  $\phi 1$ , indicating that the Address Bus, Data Bus, VMA and R/W lines are back on the bus. A single byte, 2 cycle instruction such as LSR is used for this example also. During the first cycle, the instruction Y is fetched from address M + 1. BA returns high at tBA on the last cycle of the instruction indicating the MPU is off the bus. If instruction Y had been three cycles, the width of the BA low time would have been increased by one cycle.

### Three State Control (TSC)

When the Three-State Control (TSC) line is a logic "1", the Address Bus and the R/W line are placed in a high impedance state. VMA and BA are forced low whenever TSC = "1" to prevent false reads or writes on any device enabled by VMA. While TSC is held high, the  $\phi 1$  and  $\phi 2$  clocks must be held high and low, respectively, in order to delay program execution (this is required because of the bus lines being in an indeterminate state). Since the MPU is a dynamic device, the  $\phi 1$  clock can be stopped for a maximum time  $\text{PW}_{\phi H}$  without destroying data within the MPU. TSC then can be used in a short Direct Memory Access (DMA) application.

Figure 16 shows the effect of TSC on the MPU. TSC must have its transitions at tTSE (three-state enable) while holding  $\phi 1$  high and  $\phi 2$  low as shown. The Address Bus and R/W line will reach the high impedance state at tTSD (three-state delay), with VMA being forced low. In this example, the Data Bus is also in the high impedance state while  $\phi 2$  is being held low since  $\text{DBE} = \phi 2$ . At this point in

FIGURE 15 –  $\overline{\text{Halt}}$  AND SINGLE INSTRUCTION EXECUTION FOR SYSTEM DEBUG



Note: Midrange waveform indicates high impedance state.

time, a DMA transfer could occur on cycles #3 and #4. When TSC is returned low, the MPU Address and R/W lines return to the bus. Because it is too late in cycle #5 to access memory, this cycle is dead and used for synchronization. Program execution resumes in cycle #6.

### Bus Available (BA)

The Bus Available signal will normally be in the low state; when activated, it will go to the high state indicating that the microprocessor has stopped and that the address bus is available. This will occur if the  $\overline{\text{Halt}}$  line is in the low state (Figure 15), or if the processor is in the WAIT state (Figure 14) as a result of the execution of a WAIT instruction. At such time, all three-state output drivers will go to their off state and other outputs to their normally inactive level. The processor is removed from the WAIT state by the occurrence of a maskable (mask bit 1 = 0) or non-maskable

interrupt. Note that if TSC is in the high state, Bus Available will be low.

### Data Bus Enable (DBE)

DBE is the three-state control signal for the MPU data bus and will enable the bus drivers when in the high state. This input is TTL compatible; however in normal operation, it would be driven by the  $\phi 2$  clock. During an MPU read cycle, the data bus drivers will be disabled internally. When it is desired that another device control the data bus such as in DMA applications, DBE should be held low.

If additional data setup or hold time is required on an MPU write, the DBE down time can be decreased as shown in Figure 6 (DBE  $\neq \phi 2$ ). The minimum down time for DBE is  $t_{DBE}$  as shown in Table 4, and must occur within  $\phi 1$  up time. The minimum delay from the trailing edge of DBE to the trailing edge of  $\phi 1$  is  $t_{DBED}$ . By skewing DBE with respect to E in this manner, data setup or hold time can be increased.

FIGURE 16 – THREE STATE CONTROL TIMING

